

DSS：基于共享基变换与核心空间稀疏选择的参数高效微调方法

摘要

本文档用于快速说明当前 DSS (Dynamic Sparse Selective) PEFT 方法的核心思想、数学形式、训练流程和关键工程实现。当前版本的 DSS 并不是直接在原始权重矩阵上做稀疏更新，而是先通过离线拟合得到的 shared basis 将权重更新表示到 core-space / Λ 空间，然后在线训练少量 core-space 稀疏 slot。训练过程分为两个阶段：stage1 通过 candidate probe 动态填充 elite pool；stage2 在 elite pool 填满并稳定训练后，周期性执行 prune/grow。本文档的目标不是完整论文投稿，而是让读者尽快理解该方法的设计动机、核心公式、前向反向路径、参数更新逻辑，以及公式与代码变量之间的对应关系。

1 方法定位与动机

大模型微调的核心矛盾在于：完整微调可以提供较强的表达能力，但需要更新和保存大量参数；参数高效微调 (Parameter-Efficient Fine-Tuning, PEFT) 方法只更新少量参数，但不同方法对更新空间的约束不同，因而表达能力和训练效率也不同。

LoRA 将权重增量限制为低秩形式：

$$\Delta W = BA, \quad (1)$$

其中 $B \in \mathbb{R}^{d_{out} \times r}$, $A \in \mathbb{R}^{r \times d_{in}}$, $r \ll \min(d_{out}, d_{in})$ 。这种形式简单、稳定、易于合并，但低秩结构可能限制更新空间。

Sparse Fine-Tuning / SpIEL 一类方法直接在原始权重空间中选择少量坐标进行更新。设原始权重向量化后为 θ ，稀疏更新由坐标集合 η 和对应数值 ϕ 表示，则可以写成：

$$\theta' = \theta + \text{scatter}(\eta, \phi). \quad (2)$$

这类方法的优势是更新位置更自由，但位置选择本身是离散问题；如果要全量估计所有位置的重要性，代价很高。

当前 DSS 的设计目标是折中这两类思路：

- 不使用低秩矩阵直接限制 ΔW ；
- 不直接在原始 W 空间进行稀疏选择；
- 先通过 shared basis 将更新表示到更结构化的 core-space；

- 在线训练时，只在 core-space 中维护少量动态选择的稀疏 slot；
- candidate 只做梯度探测，不影响当前前向输出；
- elite 才真正参与前向和 optimizer 更新。

因此，当前 DSS 可以概括为：

DSS 是一种 shared-basis + core-space sparse selective PEFT。它把权重更新写成 $\Delta W = A^{-1}\Delta\Lambda B^{-1}$ ，只训练 $\Delta\Lambda$ 中少量动态选择的稀疏坐标，并通过两阶段机制维护这些坐标。

2 统一符号与代码变量对应

本节优先按照当前实现中的 README 和代码变量来统一符号。目标不是形式化炫技，而是让读者可以直接把公式中的对象对应到代码里的状态。

2.1 矩阵与维度

考虑一个目标线性层，其冻结的原始权重为：

$$W \in \mathbb{R}^{d_{out} \times d_{in}}. \quad (3)$$

输入和输出分别记为：

$$x \in \mathbb{R}^{B \times T \times d_{in}}, \quad y \in \mathbb{R}^{B \times T \times d_{out}}, \quad (4)$$

其中 B 是 batch size, T 是序列长度。为了简化公式，也可以把前两维合并，把 x 看成：

$$x \in \mathbb{R}^{N \times d_{in}}, \quad (5)$$

其中 $N = BT$ 。

DSS 引入的权重增量为：

$$\Delta W \in \mathbb{R}^{d_{out} \times d_{in}}. \quad (6)$$

但 DSS 不直接训练 ΔW ，而是训练 core-space 中的稀疏矩阵：

$$\Delta\Lambda \in \mathbb{R}^{d_{out} \times d_{in}}. \quad (7)$$

通过 shared basis 的逆矩阵恢复到原始权重空间：

$$\Delta W = A^{-1}\Delta\Lambda B^{-1}. \quad (8)$$

当前代码中：

- A^{-1} 对应 `basis_A_inv`；
- B^{-1} 对应 `basis_B_inv`；
- $\Delta\Lambda$ 是 core-space 中的稀疏更新矩阵；
- ΔW 是 merge/export 时可以显式构造的原始权重空间增量。

2.2 稀疏 slot 与索引

DSS 的长期可训练参数不是一个 dense 矩阵，而是一个固定长度的一维向量：

$$c \in \mathbb{R}^K, \quad (9)$$

其中 K 是每层的稀疏预算，对应代码中的：

$$K = \text{n_frequency}. \quad (10)$$

每个 slot 的位置由 flat index 表示：

$$I \in \{0, 1, \dots, d_{out}d_{in} - 1\}^K. \quad (11)$$

代码变量对应为：

- c 对应 `coefficient`;
- I 对应 `coefficient_indices`;
- K 对应 `n_frequency`;
- 当前已经启用的 elite 数量 K_e 对应 `curr_count`。

只有前 K_e 个 slot 是当前有效 elite：

$$c_{1:K_e} = \text{coefficient}[:\text{curr_count}], \quad I_{1:K_e} = \text{coefficient_indices}[:\text{curr_count}]. \quad (12)$$

candidate 不是长期参数。它只是一批临时 probe 坐标：

$$C \in \{0, 1, \dots, d_{out}d_{in} - 1\}^{K_c}, \quad (13)$$

其中：

$$K_c = \text{candidate_size}, \quad C = \text{candidate_indices}. \quad (14)$$

2.3 flat index 与二维坐标

由于代码中 `coefficient_indices` 和 `candidate_indices` 使用 flat index，因此需要明确 flat index 与二维坐标的关系。

给定 flat index p ，其对应的 core-space 二维坐标为：

$$i = \left\lfloor \frac{p}{d_{in}} \right\rfloor, \quad j = p \bmod d_{in}. \quad (15)$$

反过来，二维坐标 (i, j) 对应的 flat index 为：

$$p = i \cdot d_{in} + j. \quad (16)$$

因此，elite slot 构成的 $\Delta\Lambda$ 可以写成：

$$\Delta\Lambda_{i,j} = \sum_{m=1}^{K_e} c_m \cdot \mathbf{1}[(i, j) = \text{unflatten}(I_m)]. \quad (17)$$

这里的 `unflatten( $I_m$ )` 表示将 flat index I_m 转为二维坐标。

数学符号	代码变量	含义
W	<code>base_layer.weight</code>	冻结的原始线性层权重
ΔW	<code>get_delta_weight()</code> 输出	原始权重空间中的 DSS 增量, 主要用于 merge/export
A^{-1}	<code>basis_A_inv</code>	shared basis 的左逆变换, 用于从 core-space 映射回输出空间
B^{-1}	<code>basis_B_inv</code>	shared basis 的右逆变换, 用于从输入空间映射到 core-space
$\Delta\Lambda$	<code>lambda_dense</code>	core-space 中的更新矩阵, 训练 forward 中由 elite slot scatter 得到
c	<code>coefficient</code>	DSS 唯一真正训练的稀疏参数向量
I	<code>coefficient_indices</code>	elite slot 在 $\Delta\Lambda$ 中的 flat index
K	<code>n_frequency</code>	每层稀疏 elite pool 的最大容量
K_e	<code>curr_count</code>	当前已经启用的 elite 数量
C	<code>candidate_indices</code>	当前临时 candidate 坐标集合
K_c	<code>candidate_size</code>	每轮 candidate batch 大小
G_Λ	<code>grad_core</code>	$\partial\mathcal{L}/\partial\Delta\Lambda$, 由 hook 捕获
T_g	<code>grad_store_steps</code>	candidate 梯度累计窗口长度

表 1: DSS 数学符号与代码变量对应表。

2.4 符号表

3 Shared Basis 与 Core-Space 表示

DSS 的核心思想之一是: 不要直接在原始权重空间中选择稀疏更新位置, 而是先为一组同功能模块拟合 shared basis, 再在 core-space 中做稀疏选择。

设同一功能组中有 L 个目标线性层, 例如所有层的 `q_proj`:

$$W_1, W_2, \dots, W_L. \quad (18)$$

离线阶段拟合两个共享矩阵 A 和 B , 使得:

$$\Lambda_l = AW_lB \quad (19)$$

在经验上更接近对角结构或带状结构。

一种直观的优化目标是降低带外能量与带内能量的比例:

$$\mathcal{L}_{basis} = \sum_{l=1}^L \frac{\|\text{outside}(AW_lB)\|_F^2}{\|\text{inside}(AW_lB)\|_F^2 + \epsilon}. \quad (20)$$

这里:

- `inside(·)` 表示保留对角线或指定带宽内的元素;
- `outside(·)` 表示带外元素;

- ϵ 是数值稳定项。

离线拟合完成后，训练阶段冻结 A^{-1} 和 B^{-1} 。在线微调时，DSS 不再训练 dense ΔW ，而是只训练 sparse $\Delta\Lambda$ ：

$$W_{eff} = W + \Delta W = W + A^{-1}\Delta\Lambda B^{-1}. \quad (21)$$

这意味着 DSS 的搜索空间从原始权重空间变成了 core-space。直观上，如果 shared basis 能够让同功能层的权重在 core-space 中呈现更集中的结构，那么在 $\Delta\Lambda$ 中选择少量稀疏坐标就可能比在原始 W 空间中随机选择坐标更有效。

4 DSS Layer 的前向传播

当前训练主路径不依赖自定义 CUDA sparse kernel，而是使用 PyTorch GEMM 与 hook。对输入 x ，base layer 的输出为：

$$y_{base} = xW^{\top}. \quad (22)$$

DSS 分支首先将输入映射到 core-space 的输入侧：

$$x_B = x(B^{-1})^{\top}. \quad (23)$$

然后由当前 elite slot 构造 core-space 稀疏更新矩阵 $\Delta\Lambda$ 。在代码中，这一步对应：

```
lambda_flat = zeros(out_features * in_features)
lambda_flat.scatter_add_(0, coefficient_indices[:curr_count], coefficient[:curr_count])
lambda_dense = lambda_flat.view(out_features, in_features)
```

数学上即：

$$\Delta\Lambda = \text{scatter}(I_{1:K_e}, c_{1:K_e}). \quad (24)$$

随后在 core-space 中计算：

$$y_{\Lambda} = x_B \Delta\Lambda^{\top}. \quad (25)$$

再将其映射回输出空间：

$$y_{\Delta} = y_{\Lambda}(A^{-1})^{\top}. \quad (26)$$

最终输出为：

$$y = y_{base} + s \cdot y_{\Delta}, \quad (27)$$

其中 s 是可选的缩放系数，对应代码中的 `delta_scale`。

将上述公式合并，可以得到：

$$\begin{aligned} y &= xW^{\top} + s \cdot \left[x(B^{-1})^{\top} \Delta\Lambda^{\top} (A^{-1})^{\top} \right] \\ &= xW^{\top} + s \cdot x (A^{-1} \Delta\Lambda B^{-1})^{\top} \\ &= x (W + s \cdot A^{-1} \Delta\Lambda B^{-1})^{\top}. \end{aligned} \quad (28)$$

这说明训练 forward 与显式构造 ΔW 再做 linear 在数学上是等价的；区别在于当前实现优先在 core-space 路径中计算，避免在训练主路径中频繁构造最终的 dense ΔW 。

4.1 Candidate 不参与前向

在前向传播中，只有 elite slot 被写入 $\Delta\Lambda$ 。candidate 坐标 C 不进入 $\Delta\Lambda$ ，因此不影响当前输出：

$$\Delta\Lambda = \text{scatter}(I_{1:K_e}, c_{1:K_e}), \quad C \notin \text{forward}(\Delta\Lambda). \quad (29)$$

这点是 DSS 语义的核心：

Candidate 是 probe-only。它只在 backward 中读取梯度信号，不参与 forward，也不会被 optimizer 更新。

5 反向传播与 Candidate Probe

虽然 candidate 不参与前向，但 DSS 仍然希望知道：如果某个 candidate 位置被加入 $\Delta\Lambda$ ，它的梯度信号是否足够大。因此，在 backward 中需要读取 candidate 对应的 core gradient。

设训练损失为 $\mathcal{L}$ 。对 core-space 更新矩阵的梯度为：

$$G_\Lambda = \frac{\partial \mathcal{L}}{\partial \Delta\Lambda} \in \mathbb{R}^{d_{out} \times d_{in}}. \quad (30)$$

当前实现会在 `lambda_dense` 上注册 hook，捕获 G_Λ 。然后对 candidate flat index 进行抽取：

$$g_C = G_\Lambda^{flat}[C], \quad (31)$$

其中 G_Λ^{flat} 表示将 G_Λ 展平成一维向量。

代码逻辑可以概括为：

```
def hook(grad_core):
    candidate_grads = grad_core.reshape(-1).index_select(0, candidate_indices)
    collect_probe_grads(candidate_grads, collect_mode)
    return grad_core
```

这里 hook 不改变原始梯度，只额外读取 candidate 的 probe gradient。

5.1 Elite 的梯度

elite slot 参与了 forward，因此其梯度由正常 autograd 自动产生：

$$\frac{\partial \mathcal{L}}{\partial c_m} = \frac{\partial \mathcal{L}}{\partial \Delta\Lambda_{I_m}^{flat}}, \quad m = 1, \dots, K_e. \quad (32)$$

因此，elite 是真正 trainable 的参数。

5.2 Candidate 的梯度

candidate 没有参与 forward，因此它没有对应的 trainable parameter，也不会进入 optimizer。但是 DSS 通过 hook 读取其位置上的 G_Λ ：

$$g_{C_j}^{(t)} = G_{\Lambda, t}^{flat}[C_j], \quad j = 1, \dots, K_c. \quad (33)$$

这些值只用于位置选择，不用于当前 step 的参数更新。

6 Stage1: 动态填充 Elite Pool

Stage1 的目标是从空的 elite pool 开始, 不断从随机 candidate 中挑选高价值位置, 直到 elite pool 填满。

初始状态为:

$$K_e = 0, \quad \text{phase} = \text{stage1}. \quad (34)$$

每一轮 stage1 选择过程包括以下步骤。

6.1 采样 Candidate

从 core-space 所有 flat index 中随机采样 candidate 集合:

$$C = \{C_1, C_2, \dots, C_{K_c}\}. \quad (35)$$

采样时需要避开已有 elite 位置:

$$C \cap I_{1:K_e} = \emptyset. \quad (36)$$

代码中通过 elite_bitset 实现该约束。

6.2 累计 Candidate Probe Gradient

在接下来的 T_g 个 optimizer step 中, candidate 集合保持不变。每一步 backward 时, hook 会记录:

$$g_C^{(t)} = G_{\Lambda, t}^{flat}[C]. \quad (37)$$

Stage1 使用 signed gradient 的均值再取绝对值作为分数:

$$s_C = \left| \frac{1}{T_g} \sum_{t=1}^{T_g} g_C^{(t)} \right|. \quad (38)$$

对应代码语义为:

```
x_mean = abs(grad_cache / grad_count)
```

需要注意, 当前 stage1 使用的是:

$$|\text{mean}(g)|, \quad (39)$$

而不是:

$$\text{mean}(|g|). \quad (40)$$

前者更强调多步梯度方向的一致性; 如果一个 candidate 的梯度正负震荡很强, 即使瞬时绝对值较大, 其 signed mean 也可能被抵消。

6.3 在线分位数阈值

DSS 使用一个在线分位数估计器来决定晋升阈值。设目标选择比例为 r , 对应代码中的 `ratio`, 则目标分位点可写为:

$$q = 1 - r. \quad (41)$$

估计器维护一个标量阈值 τ 。给定 candidate 分数 s , 使用 pinball loss 更新:

$$\ell_q(s, \tau) = \begin{cases} q(s - \tau), & s > \tau, \\ (q - 1)(s - \tau), & s \leq \tau. \end{cases} \quad (42)$$

优化该损失可以使 τ 追踪 s 的 q 分位数。代码中的 `GPUQuantileEstimator` 即维护该标量阈值, 并用内部 SGD 更新。

6.4 晋升规则

给定分数 s_C 和阈值 τ , 候选晋升集合为:

$$P = \{j \mid s_{C_j} > \tau\}. \quad (43)$$

实际晋升数量还受到每轮上下限和剩余 budget 约束。设:

$$k_{min} = \text{low}, \quad k_{max} = \text{up}, \quad K_{rem} = K - K_e. \quad (44)$$

最终晋升数量满足:

$$|P| \leq \min(k_{max}, K_{rem}). \quad (45)$$

当超过上限时, 选择分数最高的若干 candidate; 当少于下限时, 强制选择分数最高的 k_{min} 个。

晋升后, 将选中的 candidate 写入 elite pool:

$$I_{K_e+1:K_e+|P|} \leftarrow C_P, \quad (46)$$

$$C_{K_e+1:K_e+|P|} \leftarrow 0, \quad (47)$$

$$K_e \leftarrow K_e + |P|. \quad (48)$$

新晋升的 coefficient 初始化为 0, 表示新位置从无增量开始训练。

6.5 Stage1 算法

7 Stage2: 稳定训练后的 Prune/Grow

当 stage1 结束, 即:

$$K_e = K, \quad (49)$$

DSS 进入 stage2。Stage2 的目标不是继续扩大参数量, 而是在固定 budget 内周期性替换部分位置, 让 elite pool 可以适应训练过程中变化的梯度分布。

Algorithm 1 Stage1: Dynamic Elite Filling

```

1: 初始化  $K_e \leftarrow 0$ 
2: while  $K_e < K$  do
3:   随机采样 candidate 坐标  $C$ , 并保证  $C \cap I_{1:K_e} = \emptyset$ 
4:   固定  $C$ , 累计  $T_g$  个 optimizer step 的 candidate probe gradient
5:   计算  $s_C = \left| \frac{1}{T_g} \sum_{t=1}^{T_g} g_C^{(t)} \right|$ 
6:   用 pinball quantile estimator 更新阈值  $\tau$ 
7:   根据  $s_C > \tau$ 、low、up 和剩余 budget 选择晋升集合  $P$ 
8:   写入  $I_{K_e+1:K_e+|P|} \leftarrow C_P$ , 并令对应  $c \leftarrow 0$ 
9:   更新  $K_e \leftarrow K_e + |P|$ 
10: end while
11: 切换到 stage2

```

7.1 Steady Phase

Stage1 填满后, DSS 不立即 prune/grow, 而是先固定 elite pool 训练一段时间。设总训练步数为 T_{total} , 稳定期比例为 ρ_{steady} , 则:

$$T_{steady} = \lceil \rho_{steady} T_{total} \rceil. \quad (50)$$

对应代码变量为:

$$\rho_{steady} = \text{steady_stage_ratio}, \quad T_{steady} = \text{steady_phase}. \quad (51)$$

稳定期内:

- elite 正常参与 forward;
- coefficient 正常由 optimizer 更新;
- 不进行 candidate probe;
- 不执行 prune/grow。

7.2 Stage2 Candidate Window

稳定期结束后, 每隔 T_u 步执行一次更新:

$$T_u = \text{update_interval}. \quad (52)$$

每个 interval 的最后 T_g 步作为 candidate probe window:

$$T_g = \min(\text{grad_store_steps}, \text{update_interval}). \quad (53)$$

在该窗口内, DSS 采样一批新的 candidate, 并累计:

$$\mu_C = \frac{1}{T_g} \sum_{t=1}^{T_g} g_C^{(t)}, \quad (54)$$

$$v_C = \frac{1}{T_g} \sum_{t=1}^{T_g} \left(g_C^{(t)}\right)^2. \quad (55)$$

代码中分别对应:

- $\sum_t g_C^{(t)}$ 对应 `candidate_grad_sums`;
- $\sum_t (g_C^{(t)})^2$ 对应 `candidate_grad_sq_sums`。

7.3 Prune 规则

当前 stage2 的 prune 规则是: 从 elite 中选择 coefficient 绝对值最小的 k 个 slot。

设当前 elite 数值为:

$$c_1, c_2, \dots, c_K. \quad (56)$$

prune 集合为:

$$P_{drop} = \text{TopK}_{small}(|c_1|, |c_2|, \dots, |c_K|; k). \quad (57)$$

直观含义是: 如果某些 elite slot 的增量幅度长期很小, 那么它们对当前模型的贡献较弱, 可以让出位置给新的 candidate。

7.4 Grow 规则

grow 规则是: 从 candidate 中选择平均梯度绝对值最大的 k 个位置。

给定 stage2 candidate 的平均梯度 μ_C , grow 集合为:

$$P_{grow} = \text{TopK}_{large}(|\mu_C|; k). \quad (58)$$

直观含义是: 如果某些未训练位置在当前训练窗口内表现出较大的稳定梯度, 它们可能是值得加入 elite pool 的新位置。

7.5 替换操作

Stage2 使用一对一替换:

$$I_{P_{drop}} \leftarrow C_{P_{grow}}, \quad (59)$$

$$c_{P_{drop}} \leftarrow 0. \quad (60)$$

同时更新 elite membership:

$$\text{elite_bitset}[I_{old}] \leftarrow \text{False}, \quad (61)$$

$$\text{elite_bitset}[I_{new}] \leftarrow \text{True}. \quad (62)$$

当前实现中, `update_margin` 更接近未来 grow-vs-prune gate 的占位字段。也就是说, 目前 stage2 的基本逻辑是固定执行 top-k prune/grow, 而不是先判断 grow 是否显著优于 prune 后再替换。

Algorithm 2 Stage2: Periodic Prune/Grow

```

1: Stage1 填满后, 固定 elite pool 训练  $T_{steady}$  步
2: for 每个 update interval do
3:   在 interval 尾部采样 candidate 集合  $C$ 
4:   累计  $T_g$  步 candidate probe gradient
5:   计算  $\mu_C$  和  $v_C$ 
6:   从 elite 中选择  $|c|$  最小的  $k$  个 slot 作为  $P_{drop}$ 
7:   从 candidate 中选择  $|\mu_C|$  最大的  $k$  个位置作为  $P_{grow}$ 
8:   执行  $I_{P_{drop}} \leftarrow C_{P_{grow}}$ , 并令  $c_{P_{drop}} \leftarrow 0$ 
9:   局部重写 optimizer state
10: end for

```

7.6 Stage2 算法**8 Optimizer State 处理**

Stage2 动态替换 slot 后, optimizer state 的处理非常关键。普通 AdamW 对一个参数向量维护:

$$m_t, \quad v_t, \quad (63)$$

其中 m_t 是一阶动量, v_t 是二阶动量。如果直接把一个被 prune 的 slot 换成新的 grow 位置, 但保留旧动量, 就会发生状态污染: 新位置继承了旧位置的历史优化信息。

因此, DSS 使用支持 slot-wise state 的 optimizer。对每个 coefficient slot, 维护:

$$age_i, \quad m_i, \quad v_i. \quad (64)$$

其中:

- age_i 表示该 slot 已经经历了多少有效 optimizer step;
- m_i 是该 slot 的一阶动量;
- v_i 是该 slot 的二阶动量。

当 stage2 发生替换时, 只重写被替换的 slot:

$$age_{P_{drop}} \leftarrow age_{new}, \quad (65)$$

$$m_{P_{drop}} \leftarrow \hat{\mu}_{C_{P_{grow}}}, \quad (66)$$

$$v_{P_{drop}} \leftarrow \hat{v}_{C_{P_{grow}}}. \quad (67)$$

其中 $\hat{\mu}$ 和 $\hat{v}$ 来自 candidate probe window 的统计, 并根据 Adam 的 β_1, β_2 做相应的 bias counter-correction。

这样做的意义是:

- 未被替换的 elite slot 保留自己的历史 optimizer state;
- 新 grow 的 slot 不继承旧位置的动量;
- 新 grow 的 slot 也不是完全冷启动, 而是用 candidate probe 统计进行 warm start;
- 整个 coefficient 向量不需要整体清空状态。

9 训练循环时序

DSS 的 layout refresh 不应该在 backward 中立即修改参数结构, 而应该在 optimizer step 之后统一执行。推荐训练时序为:

```
loss.backward()
optimizer.step()
model.refresh_dss_layout(
    global_step=global_step,
    total_steps=total_train_steps,
    optimizer=optimizer,
    grad_accumulation_steps=grad_accumulation_steps,
)
optimizer.zero_grad()
```

该顺序的原因是:

1. backward 已经收集到当前 step 的 candidate probe gradient;
2. optimizer step 先更新当前有效 elite 参数;
3. layout refresh 再消费 candidate 统计, 决定是否晋升或 prune/grow;
4. zero grad 后进入下一步, 避免梯度和新旧 layout 混在一起。

如果在 optimizer step 之前修改 layout, 可能导致本 step 的梯度对应旧坐标, 而参数结构已经变成新坐标, 从而引入语义不一致。

10 Merge 与 Export 路径

训练主路径中, DSS 不需要显式构造最终 dense ΔW 。但是在 merge/export 时, 需要把当前 elite slots 还原到原始权重空间。

首先由 elite slots 构造 $\Delta\Lambda$:

$$\Delta\Lambda = \text{scatter}(I_{1:K_e}, c_{1:K_e}). \quad (68)$$

然后计算:

$$\Delta W = A^{-1} \Delta\Lambda B^{-1}. \quad (69)$$

merge 时执行:

$$W \leftarrow W + \Delta W. \quad (70)$$

unmerge 时执行:

$$W \leftarrow W - \Delta W. \quad (71)$$

当前代码中的 `get_delta_weight()` 可以使用更高效的 outer-product / einsum 方式从 sparse slots 直接累加得到 dense ΔW , 避免先构造完整 dense $\Delta \Lambda$ 再做两次矩阵乘法。其核心等价形式为:

$$\Delta W = \sum_{m=1}^{K_e} c_m \cdot A_{:,i_m}^{-1} \left(B_{j_m,:}^{-1} \right), \quad (72)$$

其中 $(i_m, j_m) = \text{unflatten}(I_m)$ 。

11 与相关 PEFT 方法的区别

方法	更新空间	参数结构	动态选择	shared basis
LoRA	原始权重空间	低秩矩阵 BA	否	否
Sparse FT / SpIEL	原始权重空间	稀疏 delta values + indices	是	否
LoCA / Fourier 类	频域或 DCT 空间	频率系数 + 位置	可选/部分	否
旧 DSS	原始或简单稀疏空间	elite + candidate 点	是	弱或无
当前 DSS	core-space / Λ 空间	shared-basis sparse slots	是	是

表 2: DSS 与若干 PEFT 方法的概念区别。

当前 DSS 的关键区别在于:

- 它不是低秩更新, 而是 sparse selective 更新;
- 它不是直接在 W 空间选择稀疏坐标, 而是在 shared-basis 定义的 $\Delta \Lambda$ 空间选择;
- 它不是一次性静态选择, 而是 stage1/stage2 动态维护坐标集合;
- candidate 不参与 forward, 只通过 backward hook 提供选择信号;
- stage2 替换时显式处理 optimizer state, 避免动态坐标带来的状态污染。

12 当前工程实现要点

当前实现可以分成以下几个核心文件或模块。

12.1 配置层

`config.py` 中的 `DSSConfig` 负责保存方法超参数。关键字段包括：

- `shared_basis_path`: 离线 shared basis 文件路径；
- `basis_group_map`: 功能组到 target module suffix 的映射；
- `n_frequency`: 每层 elite pool 总预算；
- `candidate_size`: 每轮 candidate batch 大小；
- `grad_store_steps`: candidate 梯度累计窗口；
- `low/up`: stage1 每轮晋升数量约束；
- `ratio`: stage1 阈值估计目标比例；
- `stage2_enabled`: 是否启用 stage2；
- `steady_stage_ratio`: stage2 前稳定期比例；
- `update_interval`: stage2 替换周期；
- `update_counts`: stage2 初始替换预算；
- `update_margin`: 未来 grow-vs-prune gate 的占位字段。

12.2 模型注入层

`model.py` 中的 `DSSModel` 负责：

1. 加载 shared basis；
2. 根据 `basis_group_map` 判断目标模块属于哪个功能组；
3. 将目标 `nn.Linear` 或兼容模块替换成 `DSSLinear`；
4. 冻结 base model, 只让 `coefficient` 可训练；
5. 提供 `refresh_dss_layout(...)` 作为外部训练循环入口。

12.3 核心层

`layer.py` 是 DSS 的核心实现，主要负责：

- 保存每层 elite/candidate/runtime 状态；
- 执行 pure PyTorch GEMM forward；
- 在 `lambda_dense` 上注册 hook 读取 candidate probe gradient；

- 执行 stage1 晋升;
- 执行 stage2 prune/grow;
- 调用 optimizer state rewrite。

12.4 优化器

`optimizer.py` 中的 `DSSAdamW` 负责 slot-wise AdamW 更新, 并支持 stage2 后的局部 optimizer state 重写。

13 当前局限与后续方向

当前 DSS 实现已经具备完整的 shared-basis core-space sparse selective 训练流程, 但仍有一些可以继续改进的方向。

13.1 当前训练路径仍构造 dense lambda

训练 forward 中目前会构造 `dense lambda_dense`。虽然它是在 core-space 中构造, 而不是最终的 dense ΔW , 但当 $d_{out}d_{in}$ 很大时, 仍然存在额外开销。

后续可以考虑设计 DSS 专用 `linear_sd` operator, 使其直接用:

$$(x_B, c_{1:K_e}, I_{1:K_e}, C) \quad (73)$$

完成 sparse core forward 和 backward candidate probe, 从而减少每步 scatter dense matrix 的开销。

13.2 Grow-vs-Prune Gate 尚未启用

当前 stage2 的 `update_margin` 主要是占位字段。更严格的设计应该先比较 grow candidate 的收益是否显著高于 prune slot 的保留价值, 再决定是否替换。例如可以设计:

$$|\mu_{grow}| > \alpha |c_{prune}| + \gamma \quad (74)$$

或其他 margin gate。这样可以避免在 stage2 后期进行低收益替换。

13.3 Shared Basis 质量需要诊断

DSS 的有效性依赖 shared basis 是否真的让 core-space 结构更适合稀疏选择。因此后续需要记录:

- $AW_l B$ 的带内/带外能量比例;
- A 、 B 或其逆矩阵的条件数;
- $\Delta\Lambda$ 中 elite 分布是否集中;

- 不同功能组的 basis 是否质量差异较大。

如果某些功能组的 shared basis 质量不稳定, 可以考虑 fallback 到 identity basis、分组更细的 basis, 或重新拟合。

14 简短总结

当前 DSS 的核心可以浓缩为以下几点:

1. 对每个目标线性层, DSS 不直接训练 ΔW , 而是训练 core-space 稀疏更新 $\Delta \Lambda$ 。
2. 原始权重空间和 core-space 之间通过离线 shared basis 连接: $\Delta W = A^{-1} \Delta \Lambda B^{-1}$ 。
3. 在线训练中, `coefficient` 是唯一真正训练的 DSS 参数; `coefficient_indices` 记录其在 $\Delta \Lambda$ 中的位置; `curr_count` 表示当前 elite 数量。
4. `candidate` 由 `candidate_indices` 表示, 不参与 forward, 也不进入 optimizer, 只在 backward 中通过 hook 读取 probe gradient。
5. Stage1 用 candidate probe gradient 和在线 quantile threshold 动态填充 elite pool。
6. Stage2 在稳定训练后周期性执行 prune/grow, 并对新 grow slot 进行 optimizer state warm start。
7. 训练主路径使用 PyTorch GEMM + hook, merge/export 时再显式恢复 ΔW 。

因此, DSS 可以被理解为一种在 shared-basis core-space 中执行的动态稀疏参数高效微调方法。它试图同时获得 selective fine-tuning 的自由坐标选择能力、shared-basis 的结构化归纳偏置, 以及动态 sparse training 的持续修正能力。